Larkspur Commerce Production Operations Runbook

Contents

1. Purpose and scope **6**

1.1 Purpose of this runbook 6

1.2 Scope 6

1.3 Audience and how to use this document 6

1.4 Conventions used in this document 6

1.5 Ownership and review of this runbook 7

2. Platform architecture **8**

2.1 Overview 8

2.2 Service inventory 8

2.3 Service descriptions 8

2.4 Dependency map 10

2.5 Hosts and environments 10

2.6 Observability 11

3. Incident management **12**

3.1 What counts as an incident 12

3.2 Severity definitions 12

3.3 Response times and update cadence 12

3.4 Roles 12

3.5 Declaring an incident 13

3.6 Running the incident 13

3.7 Changing severity . . . 13

3.8 Closing an incident . . . 14

3.9 Handoff between shifts . . . 14

4. Escalation and on-call **15**

4.1 On-call principles . . . 15

4.2 Contacts . . . 15

4.3 Escalation path . . . 15

4.4 Choosing whom to page . . . 15

4.5 When the on-call is unreachable . . . 16

4.6 On-call hygiene . . . 16

5. Database operations **17**

5.1 Health checks . . . 17

5.2 Disk pressure . . . 18

5.3 Replication slots . . . 19

5.4 Planned switchover . . . 20

5.5 Unplanned failover . . . 21

6. Deployment and rollback **23**

6.1 Deployment model . . . 23

6.2 Approvals . . . 23

6.3 Canary releases . . . 23

6.4 Pre-flight checks . . . 23

6.5 Rollback policy . . . 24

6.6 Version selection . . . 24

6.7 Configuration rollback . . . 25

6.8 Forward fix . . . 25

6.9 Failed releases . . . 25

7. Service-specific runbooks 26

7.1 payments-api . . . 26

7.2 auth-service . . . 27

7.3 checkout-web . . . 27

7.4 orders-api . . . 28

7.5 session-cache . . . 29

8. Post-failover recovery and service restart sequencing 30

8.1 Why order matters . . . 30

8.2 Restart order . . . 30

8.3 Procedure . . . 30

8.4 Verification after each stage . . . 31

8.5 If a stage will not become healthy . . . 31

8.6 Services that are not restarted . . . 31

8.7 Closing out recovery . . . 31

9. Customer communications and SLA credits 32

9.1 Principles . . . 32

9.2 Who communicates . . . 32

9.3 Response times for customers . . . 32

9.4 Message structure . . . 32

9.5 Handling escalated customers . . . 32

9.6 SLA tiers and credits . . . 32
9.7 Claiming and approving credits . . . 33
9.8 After the incident . . . 33
10. Security operations **34**
10.1 Principles . . . 34
10.2 Secret handling . . . 34
10.3 Log hygiene . . . 34
10.4 Prompt-injection note for AI tools . . . 35
10.5 Access control . . . 35
10.6 Suspected compromise . . . 35
11. Post-incident review **36**
11.1 Purpose . . . 36
11.2 When reviews are held . . . 36
11.3 Blameless approach . . . 36
11.4 Preparing for the review . . . 36
11.5 Review structure . . . 36
11.6 Action items . . . 36
11.7 Sharing the outcome . . . 37
12. Glossary and reference tables **38**
12.1 Glossary . . . 38
12.2 Service ownership . . . 38
12.3 Historical incidents . . . 39
12.4 Where each topic lives . . . 39

12.5 Quick command reference 39

12.6 Closing note 40

1. Purpose and scope

1.1 Purpose of this runbook

This runbook is the working reference for anyone who operates the Larkspur Commerce production platform. It describes how the platform is put together, how incidents are declared and run, how databases are kept healthy, how releases are shipped and reversed, and how customers are kept informed when something goes wrong. It is written for engineers who are on call, for Incident Commanders, for database administrators, and for the support and customer success staff who sit next to them during an outage.

The document is intentionally practical. Each procedure is written so that a tired engineer at three in the morning can follow it without having to remember context. Where a decision has to be made, the decision is presented as a table or as a short list of criteria. Where a command has to be run, the command is given together with the output you should expect to see. If the output you see differs from the expected output, stop and consult the owning team before continuing.

The runbook is the source of truth for operational policy. When a chat message, a ticket comment or a verbal instruction conflicts with this document, the document wins unless the Incident Commander explicitly records a deviation in the incident channel and states the reason.

1.2 Scope

The runbook covers the production environment of the Larkspur Commerce platform in the eu-west-1 region. It covers the eleven platform components listed in chapter 2, the two PostgreSQL clusters that sit under them, the session cache, the deployment pipeline, and the supporting processes for incident handling, communication and security.

The runbook does not cover the development, test or staging environments, except where a procedure explicitly refers to them. It does not cover internal corporate systems. It does not replace the detailed design documents that each service team maintains; those documents describe how a service works internally, while this runbook describes how to operate it safely under pressure.

1.3 Audience and how to use this document

Different readers will use different chapters. The table below gives a starting point.

Reader	Start with	Then read
New on-call engineer	Chapter 3	Chapters 4, 6, 7
Incident Commander	Chapter 3	Chapters 4, 8, 9, 11
Database administrator	Chapter 5	Chapters 8, 12
Customer success and support	Chapter 9	Chapters 3, 11
Security on-call	Chapter 10	Chapters 3, 4

Procedures are numbered. Perform the steps in order and do not skip a step because it looks unnecessary. If a step cannot be performed, write down why in the incident channel before you continue.

1.4 Conventions used in this document

Service names are written in lower case with hyphens, for example payments-api. Host names are written as they appear in the inventory, for example pgdb-01. Commands are shown in short code blocks and are written for a POSIX shell on a bastion host. Placeholders appear in angle brackets, for example `<service>`.

Times are expressed in UTC unless stated otherwise. Durations are wall-clock durations measured from the moment the relevant event is recorded in the incident channel. "Healthy" always means that the service passes its own health endpoint and its dashboards show error rate and latency within the normal band for that version.

1.5 Ownership and review of this runbook

The platform team owns this document. Every chapter has a technical reviewer from the owning team, and the whole document is reviewed after every SEV1 and at least once per quarter. Changes are made through the normal pull request process for the operations repository. A change that alters a threshold, a contact or a policy requires approval from the team that owns the affected area.

2. Platform architecture

2.1 Overview

Larkspur Commerce runs an online checkout and payments platform for retail merchants. A shopper reaches the storefront through the edge gateway, signs in through the authentication service, builds a basket in the checkout web application, and pays through the payments API. Orders are recorded by the orders API, and customers are notified by the notification service once an order or a payment changes state.

The platform is made up of eleven components. Eight are application services, one is a Redis-based cache, and two are PostgreSQL clusters. Every component has a tier, an owning team and a set of declared dependencies. Tier determines how quickly an incident is escalated and how much change risk is tolerated.

Tier	Meaning	Typical examples
1	Customer-facing path or money movement; an outage stops revenue	edge-gateway, checkout-web, auth-service, payments-api, ledger-service, session-cache, payments-db, identity-db
2	Important but with a degraded mode; failures delay or retry work	orders-api, fraud-scorer
3	Background or asynchronous; failures are tolerated for a short time	notification-service

2.2 Service inventory

Service	Tier	Owner	Kind	Instances
edge-gateway	1	platform-team	gateway	6
checkout-web	1	storefront-team	web	8
auth-service	1	identity-team	api	6
payments-api	1	payments-team	api	10
orders-api	2	orders-team	api	6
ledger-service	1	payments-team	api	4
fraud-scorer	2	risk-team	api	4
notification-service	3	platform-team	worker	3
session-cache	1	platform-team	redis	3
identity-db	1	dba-team	postgres	2
payments-db	1	dba-team	postgres	2

The instance counts above are the normal steady-state counts. A service running with fewer healthy instances than this is in a degraded capacity state even if its error rate looks normal, and the owning team should be told.

2.3 Service descriptions

2.3.1 edge-gateway

The edge gateway terminates TLS, applies rate limits and routes traffic to checkout-web and auth-service. It holds no business state. It is operated by the platform team and runs six instances spread across availability zones. A gateway problem looks like a sudden drop in total request volume combined with healthy upstream services.

2.3.2 checkout-web

The checkout web application renders the basket and checkout pages and calls payments-api, orders-api and auth-service on behalf of the shopper. It is owned by the storefront team. It is stateless with respect to databases: it keeps no database connections of its own and relies on the services behind it.

2.3.3 auth-service

The authentication service issues and validates sessions and tokens. It depends on session-cache for fast session lookups and on identity-db for durable identity data. It is owned by the identity team. Because every authenticated request passes through it, it is a tier 1 dependency of the entire storefront.

2.3.4 payments-api

The payments API accepts payment requests, calls the fraud scorer, records movements through the ledger service and writes payment state to payments-db. It is owned by the payments team and is the most sensitive service on the platform. It uses a connection pool to payments-db, and the size of that pool is a controlled setting (see section 7.1).

2.3.5 orders-api

The orders API records and serves order state and queries payment status. It depends on payments-db. It is tier 2 because checkout can accept an order into a retry queue for a short period when orders-api is slow, but a long outage leaves orders in an unknown state and creates reconciliation work.

2.3.6 ledger-service

The ledger service is the double-entry record of money movement. It depends on payments-db and is owned by the payments team. It is a hard dependency of payments-api, because accepting payments without a ledger would create unrecorded money movement.

2.3.7 fraud-scorer

The fraud scorer returns a risk score for a payment. It has no declared dependencies and is owned by the risk team. When it is unavailable, payments-api follows the documented fallback of accepting low-value payments and holding high-value ones for review.

2.3.8 notification-service

The notification service is a background worker that sends emails and webhook callbacks when order and payment states change. It reads from payments-db. It is tier 3 and is deliberately started last during recovery so that it does not flood customers with duplicate messages while upstream services are still settling.

2.3.9 session-cache

The session cache is a three-node Redis deployment holding live session data. It has no dependencies. Losing it does not lose identity data, but every session must then be rebuilt from identity-db, which can create a surge of load on that database.

2.3.10 identity-db

identity-db is a two-node PostgreSQL cluster owned by the DBA team. It stores accounts, credentials metadata and long-lived identity records. It is separate from payments-db on purpose, so that a payments database event does not affect sign-in.

2.3.11 payments-db

payments-db is a two-node PostgreSQL 16 cluster managed with Patroni. The primary is pgdb-01 and the standby is pgdb-02. The standby receives streaming replication from the primary. Payments, ledger entries, orders and notification state all live in this cluster, which makes it the single most important stateful component on the platform.

2.4 Dependency map

Dependencies are directional: the service on the left needs the service on the right in order to work correctly.

Service	Depends on
edge-gateway	checkout-web, auth-service
checkout-web	payments-api, orders-api, auth-service
auth-service	session-cache, identity-db
payments-api	payments-db, ledger-service, fraud-scorer
orders-api	payments-db
ledger-service	payments-db
fraud-scorer	none
notification-service	payments-db
session-cache	none
identity-db	none
payments-db	none

Reading the map from the bottom up tells you the blast radius of a failing component. A problem in payments-db touches ledger-service, payments-api, orders-api and notification-service directly, and touches checkout-web and edge-gateway indirectly. A problem in session-cache touches auth-service directly and, through it, everything that requires a signed-in shopper.

2.5 Hosts and environments

The two payments-db hosts are pgdb-01, the primary, at 10.20.4.15, and pgdb-02, the standby. Connection pooling in front of payments-db is provided by pgbouncer-payments. Application services never connect to the database hosts directly; they connect through the pooler.

Production runs in eu-west-1. Service instances are spread across three availability zones. Deployments are performed by the deploy bot, which records each release as a deployment record with an identifier such as dep-7702, a version, a previous version, a status, an author and a ticket.

2.6 Observability

Every service exposes a health endpoint, a readiness endpoint and a metrics endpoint. The standard dashboards show request rate, error rate, p95 latency, saturation and, where relevant, connection pool use. Alerts are routed through the paging system to the on-call engineer of the owning team.

The signals most useful in the first minutes of an incident are listed below.

Signal	Where to look	What it tells you
p95 latency	service dashboard	Slow dependency or saturation
Error rate	service dashboard	Hard failures versus slowness
Pool waiters	payments-api dashboard	Database connection starvation
Cache hit ratio	auth-service dashboard	Session loss or configuration problem
Disk used percent	payments-db dashboard	Storage pressure on pgdb-01
Replication lag	payments-db dashboard	Standby freshness
Ready instances	checkout-web dashboard	Capacity during a release

3. Incident management

3.1 What counts as an incident

An incident is any unplanned event that degrades, or is about to degrade, the service customers receive. A failing release, a database trending toward a full disk, a spike of login failures and a customer escalation about failed payments are all incidents. A planned maintenance window that behaves as designed is not.

If you are unsure whether something is an incident, treat it as one and declare it. It is far cheaper to close an unnecessary incident than to start late on a real one.

3.2 Severity definitions

Severity is chosen by customer impact, not by how interesting the technical cause is. Choose the higher severity when two levels seem to fit, and lower it later when the facts allow.

Severity	Definition	Examples
SEV1	Customers cannot complete payments or sign in, or data is at risk	Checkout failing for a large share of shoppers; payments database unavailable
SEV2	Significant degradation with a workaround or limited scope	Payments slow but succeeding; one tier 1 service at reduced capacity
SEV3	Minor degradation, no immediate customer harm	Background worker lagging; a single non-critical alert flapping
SEV4	Cosmetic or low-risk issue found during operation	Dashboard error; documentation gap; noisy but harmless alert

3.3 Response times and update cadence

The response targets below start when the page is sent or the incident is reported, whichever is earlier.

Severity	Acknowledge within	Status updates
SEV1	5 minutes	Every 30 minutes
SEV2	15 minutes	Every 60 minutes
SEV3	4 hours	At start and at resolution
SEV4	Next business day	At resolution

Acknowledgement means a named human has accepted the page and written their name in the incident channel. An automated acknowledgement does not count. Status updates are posted to the incident channel and to the internal status page, and they state what is known, what is being done and when the next update will be posted, even if nothing has changed.

3.4 Roles

3.4.1 Incident Commander

The Incident Commander (IC) owns the incident. The IC decides severity, assigns roles, approves risky actions, keeps the timeline and decides when the incident is over. The IC does not debug. If the IC finds themselves reading logs, they should hand the IC role to someone else and take a technical role instead.

3.4.2 Technical lead

The technical lead is usually the on-call engineer of the owning service. The technical lead forms hypotheses, directs diagnostic work and proposes remediation to the IC. There is one technical lead per workstream, and a large incident may have several.

3.4.3 Communications lead

The communications lead writes the status updates, keeps the status page current and coordinates with customer success so that customers hear one consistent story. The communications lead may be the IC on a small incident, but on a SEV1 a separate person should take the role.

3.4.4 Scribe

The scribe records the timeline in the incident channel: when alerts fired, who acknowledged, what was tried, what changed and what the result was. A good timeline is the foundation of the post-incident review.

3.5 Declaring an incident

1. Open an incident in the incident tool and give it a short, factual title.
2. Choose a provisional severity using section 3.2.
3. Create or join the incident channel named after the incident identifier.
4. Post the first message: what you see, since when, and who is working on it.
5. Page the Incident Commander rota for any SEV1 or SEV2. Contact details are in chapter 4.
6. The IC assigns the technical lead, communications lead and scribe.

3.6 Running the incident

The IC runs a short loop for the life of the incident: establish what is happening, decide what to try, assign it, wait for the result, and record it. Keep the loop tight. Prefer actions that are quick to reverse. A rollback of a release made in the last day is usually the fastest safe action, and chapter 6 describes when it is the default.

Avoid parallel risky changes. Two engineers changing two things at once make it impossible to know what helped. The IC serialises risky changes and asks for explicit confirmation before each one.

3.7 Changing severity

Severity may be raised at any time by anyone with a good reason. It may be lowered only by the IC, and the reason is written in the incident channel. When severity changes, the cadence of status updates changes immediately to match the new level.

3.8 Closing an incident

An incident is resolved when customer impact has ended and the metrics have been in the normal band long enough to be confident. It is closed when the post-incident review has been scheduled and any immediate follow-up has an owner. Chapter 11 describes the review.

3.9 Handoff between shifts

A long incident outlasts a shift. The outgoing IC gives the incoming IC a verbal summary and points to the timeline. The handoff is recorded in the channel with both names. Never hand off silently, and never leave without confirmation that the next person has accepted.

4. Escalation and on-call

4.1 On-call principles

Every tier 1 and tier 2 service has a named on-call engineer at all times. On-call engineers carry a pager, are able to reach a bastion session within the response time, and know where this runbook lives. A page is a request for action, not a notification: if you receive one, acknowledge it or hand it on.

4.2 Contacts

Contacts are listed by role and by internal extension. Use the paging system first; use the extension only when paging has failed or when the person is already on the incident.

Role	Name	Internal extension	Notes
Payments on-call	Asha Menon	ext. 4101	payments-api, ledger-service, fraud-scorer interface
DBA lead	Daniel Okafor	ext. 4102	payments-db, identity-db, pgbouncer
Identity on-call	Mei Tanaka	ext. 4103	auth-service, session-cache interface
Incident Commander rota	ic-primary	ext. 4100	Page the rota, not an individual
Security on-call	Rohan Verma	ext. 4104	Secrets, access, suspected compromise
Engineering Duty Manager	Helena Brandt	ext. 4001	Escalation of last resort

Names are the current holders of each role. The rota in the paging system is authoritative; if the name here is out of date, trust the pager and send a correction to the platform team.

4.3 Escalation path

Escalation is automatic for the highest severity and manual for the rest.

1. The alert pages the on-call engineer of the owning service.
2. If the page is not acknowledged within 15 minutes, a SEV1 page escalates automatically to the Engineering Duty Manager, Helena Brandt.
3. The Incident Commander rota is paged in parallel for SEV1 and SEV2.
4. The IC may page any additional specialist; specialists are expected to respond.

For lower severities, the on-call engineer escalates by judgment: if you are stuck for longer than you would be comfortable explaining, ask for help.

4.4 Choosing whom to page

Symptom	Page first	Also consider
Payment latency or errors	Payments on-call	DBA lead

Symptom	Page first	Also consider
Login failures, forced logouts	Identity on-call	Platform team
Database disk, replication, failover	DBA lead	Incident Commander
Suspected leaked credential or abuse	Security on-call	Incident Commander
Whole-platform outage	Incident Commander rota	Engineering Duty Manager
Checkout capacity loss during release	Storefront team on-call	Platform team

4.5 When the on-call is unreachable

If the on-call engineer does not answer after two attempts through the pager, call the extension. If there is still no answer, page the secondary on-call for that team. If the secondary also does not answer, the Incident Commander pages the team lead. Record every attempt in the incident channel with the time.

4.6 On-call hygiene

Before a shift starts, check that your pager works, that your access to the bastion and dashboards is valid, and that you have read any handoff notes. At the end of a shift, write a handoff note listing open alerts, risky changes in flight and anything that looked unusual. An on-call engineer who is exhausted should say so; the rota has a secondary for that purpose.

5. Database operations

5.1 Health checks

5.1.1 Routine checks

The DBA team reviews database health daily and the on-call engineer reviews it at the start of each shift. The checks below apply to payments-db; the same checks apply to identity-db with the host names changed.

Check	Healthy	Investigate when
Primary reachable	pgdb-01 accepts connections	Any connection failure
Standby streaming	pgdb-02 state is streaming	State is anything else
Replication lag	Well under 1 second	Sustained growth
Disk used	Below warning threshold	See section 5.2
Replication slots	All active	See section 5.3
Connections	Within the normal band	Sudden rise or collapse
Long transactions	None older than 5 minutes	Any long idle-in-transaction

5.1.2 Checking cluster state

Run the Patroni cluster listing from a bastion host.

```
patronictl list payments-db
```

A healthy cluster shows one leader and one replica, both running, with the replica lag column near zero.

```
+ Cluster: payments-db ---+---------+---------+----+-----------+
| Member  | Host          | Role    | State   | TL | Lag in MB |
| pgdb-01 | 10.20.4.15    | Leader  | running | 12 |           |
| pgdb-02 | 10.20.4.16    | Replica | running | 12 |         0 |
+---------+---------------+---------+---------+----+-----------+
```

If the replica is missing, stopped or showing a growing lag, do not attempt a switchover or failover. Establish why first.

5.1.3 Checking replication

On the primary, query the replication view.

```
SELECT application_name, state, replay_lag
FROM pg_stat_replication;
```

Expected output is one row for pgdb-02 with state streaming and a replay lag measured in milliseconds. A missing row means the standby is not connected.

5.1.4 Checking connections

Compare current connections with the baseline for the time of day. A fall in connections combined with rising latency in the application usually means that the pooler or the application is failing to connect, not that the database is idle. A sharp rise usually means a retry storm and should be

investigated at the application layer first.

5.2 Disk pressure

5.2.1 Why disk pressure matters

PostgreSQL retains write-ahead log (WAL) segments until they are no longer needed by checkpoints, archiving or replication. When something holds WAL back, such as an inactive replication slot, the volume fills quietly. A full volume on the primary stops all writes and therefore stops all payments. Disk pressure is a slow-moving incident that must be acted on before it becomes a fast-moving one.

5.2.2 Thresholds and actions

Level	Disk used	Required action
Warning	80%	Open an incident at SEV3, find the cause of growth, notify the DBA team
Critical	90%	Raise to SEV2, page the DBA lead, begin remediation
Emergency	95%	Stop batch jobs and page the DBA lead immediately; treat as SEV1

At the emergency level, stop batch jobs and page the DBA lead. Batch jobs that write heavily to payments-db, such as settlement runs and reporting extracts, must be paused before any other step, because they consume the remaining space fastest.

Never delete WAL files manually. Removing segments by hand from the data directory can make the primary unable to restart and can corrupt the standby. WAL is only ever released by PostgreSQL itself, after its cause of retention has been removed.

5.2.3 Diagnosing the cause

1. Confirm the figure on the host, not only on the dashboard.
2. Measure how much space the WAL directory occupies compared with the data files.
3. Check replication slots (section 5.3) for any inactive slot.
4. Check that archiving, if enabled, is succeeding.
5. Look for a runaway table or an unusually large transaction.

Confirm usage with the operating system.

```
df -h /var/lib/postgresql
du -sh /var/lib/postgresql/16/main/pg_wal
```

If the WAL directory accounts for most of the used space and the data files have not grown, WAL retention is the cause, and in most cases the cause is a replication slot.

5.2.4 Remediation

Remove the cause of retention, not the symptom. If an inactive slot is the cause, follow section 5.3. Once the slot is gone, a checkpoint allows PostgreSQL to recycle the retained segments. Trigger a checkpoint and then re-measure.

```
CHECKPOINT;
```

Expected result is a drop in the WAL directory size over the following minutes and a corresponding fall in disk used. The historical incident INC-0862 followed this exact pattern: an inactive analytics slot retained WAL, the slot was dropped, and a checkpoint freed 160 GB.

If the cause is not WAL, escalate to the DBA lead to decide on vacuum, table maintenance or volume expansion. Volume expansion is an online operation but must be approved by the DBA lead.

5.2.5 Verification

After remediation, confirm that disk used is falling, that the standby is still streaming, and that no alert remains active. Keep the incident open until usage is below the warning level and the cause is documented.

5.3 Replication slots

5.3.1 What a replication slot is

A replication slot tells PostgreSQL to keep WAL for a consumer until that consumer has read it. The standby uses a slot, and so does the analytics replica. A slot is useful because the consumer never misses data, but it is dangerous when the consumer stops: PostgreSQL keeps WAL for it indefinitely.

5.3.2 When an inactive slot is an incident

An inactive slot that is older than 6 hours is an incident. Open a ticket at SEV3 at the first alert and raise it if disk usage is already rising. The 6-hour threshold was set after INC-0862 so that the alert fires well before the retained WAL becomes a threat to the volume.

5.3.3 Checking slots

Query the slot view on the primary.

```
SELECT slot_name, active, restart_lsn, wal_status
FROM pg_replication_slots;
```

Expected output lists the standby slot as active and the analytics slot as active. A row with active equal to false whose restart_lsn is not advancing is an inactive slot. Note the time at which it became inactive from the database log or the alert.

5.3.4 Dropping a slot

Dropping a slot is irreversible for its consumer. The analytics replica cannot resume from a dropped slot and must be rebuilt. Therefore, drop the analytics slot only after the analytics owner, the data-platform team, confirms in the incident channel that the replica can be rebuilt.

```
SELECT pg_drop_replication_slot('analytics_slot');
```

Expected result is a single empty row and the slot disappearing from the view. The analytics replica rebuild takes about 4 hours, during which analytics dashboards are stale. Tell the data-platform team the expected duration before you drop the slot, so that they can warn their consumers.

Never drop the slot that belongs to the standby pgdb-02 to recover disk space. That would force a full rebuild of the standby and remove the failover target at the moment the cluster is under stress.

5.3.5 Preventing recurrence

Keep the inactive-slot alert enabled. Ask the data-platform team to monitor their consumer and to tell the DBA team before long maintenance. Review all slots in the weekly database review.

5.4 Planned switchover

5.4.1 When to use a switchover

A switchover is a controlled role exchange between the primary and the standby, performed while both nodes are healthy. Use it for maintenance of the primary host, for operating-system patching and for planned storage work. It causes a short interruption of writes, so it is scheduled in a low-traffic period and announced in advance.

5.4.2 Preconditions

Condition	Required
Both nodes running	Yes
Standby streaming and lag near zero	Yes
No inactive replication slots	Yes
No release in progress for payments services	Yes
Change approved and announced	Yes
DBA on-call available for the whole window	Yes

5.4.3 Procedure

1. Announce the window in the operations channel and on the status page.
2. Confirm cluster health with `patronictl list payments-db`.
3. Pause batch jobs that write to the database.
4. Run the switchover and answer the prompts with the intended new leader.
5. Watch the cluster listing until the new leader shows as running and the old leader rejoins as a replica.
6. Confirm that the pooler has reconnected and that applications are healthy.
7. Resume batch jobs.

```
patronictl switchover payments-db --leader pgdb-01 \
  --candidate pgdb-02
```

Expected output asks for confirmation, then reports a successful switchover to the candidate. Applications may log a brief burst of connection errors while the pooler reconnects; this should clear within a minute.

5.4.4 Verification and rollback

Check the cluster listing, replication lag on the new standby, error rates for payments-api and orders-api and the pool metrics. If the new leader is unhealthy, perform a second switchover back

to the original node once it is confirmed healthy. If neither node is healthy, treat the situation as an unplanned failover and move to section 5.5.

5.5 Unplanned failover

5.5.1 When to fail over

Fail over only when the primary pgdb-01 is down, unreachable or so damaged that recovery in place would take longer than a failover. A slow database, a full disk that can be cleared, or a misbehaving application is not a reason to fail over; each of those has a cheaper remedy elsewhere in this chapter.

5.5.2 Authority and targets

Failover requires the DBA on-call plus Incident Commander approval. Neither may approve alone, and the approval is recorded in the incident channel before the command is run. The target recovery time objective is 15 minutes and the recovery point objective is 30 seconds, which means that no more than 30 seconds of committed writes may be lost.

5.5.3 Procedure

1. Confirm that pgdb-01 is truly unavailable from two independent vantage points.
2. Confirm that replication lag on pgdb-02 is under 5 seconds. If it is not, tell the Incident Commander the likely data loss and wait for the decision.
3. Obtain approval from the DBA on-call and the Incident Commander.
4. Announce the failover in the incident channel.
5. Run the failover and fail over to standby pgdb-02.
6. Watch the cluster listing until pgdb-02 is the leader and running.

```
patronictl failover payments-db --candidate pgdb-02
```

Expected output asks for confirmation and then reports that pgdb-02 was promoted. Run the cluster listing again to confirm one leader and a timeline number one higher than before.

5.5.4 Services that must restart

After the new leader is promoted, some services hold stale connection state and must be restarted. The services that must restart are ledger-service, payments-api, orders-api and notification-service.

Two services do not need a restart. auth-service does not need a restart because it uses identity-db, a separate cluster that the failover did not touch. checkout-web does not need a restart because it holds no database connections.

The order in which the restarts must happen is critical and is not repeated here. Follow chapter 8 exactly.

5.5.5 After the failover

Once the services are healthy, rebuild the failed node as a replica when it is safe to do so, and confirm that replication is streaming again. Do not return to the former primary role by a second failover until a planned switchover has been scheduled. Record the timeline, the amount of data at

risk and the actual recovery time for the post-incident review.

6. Deployment and rollback

6.1 Deployment model

All production changes to services are made by the deploy bot from a reviewed pull request. Each release produces a deployment record with an identifier, the service, the new version, the previous version, a time, an author, a status and a ticket. The status is one of in-progress, completed or failed. A completed status means that the deployment finished mechanically; it does not mean the version is healthy.

Configuration is versioned together with the release. A deployment record includes a configuration diff listing every setting that changed. Reviewers must read the diff, not only the code.

6.2 Approvals

Change type	Approval needed
Routine release of a tier 2 or tier 3 service	Service owner reviewer
Routine release of a tier 1 service	Service owner reviewer plus a second engineer
Change to a controlled setting (section 7)	Service owner plus evidence listed in the section
Emergency rollback	On-call engineer of the owning service
Database change	DBA lead

Releases are not made in the final hour of a working day, during a declared incident affecting the same service, or while another release of a dependent service is still in its canary.

6.3 Canary releases

Every tier 1 release is rolled out as a canary. A small share of traffic goes to the new version, health is observed, and the share is increased in stages. Each stage has an observation period during which the release is judged against the previous version using error rate, p95 latency, saturation and the readiness status of new instances.

The default stages and observation periods are set per service and are described in the service runbooks in chapter 7. The deploy bot promotes automatically when the criteria are met and aborts automatically when a hard failure rule is hit. An engineer may also abort at any time.

A canary must never be promoted by overriding a failing check. If a check is wrong, fix the check in a separate change.

6.4 Pre-flight checks

Before a release starts, the pipeline runs pre-flight checks. They verify that the artifact exists and is signed, that the configuration diff has been reviewed, that new configuration values are within their allowed ranges, and that any new upstream host resolves in DNS from the target network. A failed pre-flight check blocks the release.

The DNS check exists because of INC-0755, where a canary stalled for lack of a record for a new upstream host. The range check on configuration values exists because of INC-0917 and INC-0801.

6.5 Rollback policy

6.5.1 Default response

Rollback is the default response to a SEV1 or SEV2 regression within 24 hours of a release. If a regression appears within that window, the first question is not what is wrong, but whether the release can simply be reversed. Diagnosis continues after the service has recovered.

6.5.2 Approval

Only an on-call engineer may approve a rollback. The Incident Commander may recommend one but cannot approve it on behalf of the service. This keeps responsibility with the person who knows the service best and who will be woken if the rollback goes wrong.

6.5.3 Selecting the target version

Choose the target as the last version with status completed that was healthy for at least 24 hours in production. Healthy is judged from the dashboards and from the deployment history, not from memory. The selection procedure is in section 6.6.

6.5.4 Service-specific conditions

Rollback of payments-api requires checking that no schema migration ran as part of the release you are reversing. If a migration ran, do not roll back; use the forward-fix path described in section 6.8, because older code may not understand the new schema.

The rollback duration for payments-api is about 6 minutes from the start of the command to all instances being healthy. Include this in your estimate when you tell the Incident Commander how long recovery will take.

6.5.5 Rollback command and verification

```
deployctl rollback <service> --to <version>
```

Expected output lists the instances being replaced in batches and ends with all instances ready. Verify against the dashboards that error rate and latency return to the band seen on the target version, and keep watching for the length of one full observation period.

6.6 Version selection

Selecting the right target is the most common source of mistakes during a rollback. Work through the steps below.

1. List recent deployments of the service from the deployment history, newest first.
2. Discard any whose status is not completed.
3. For each remaining version, check from the dashboards how long it ran in production and how it behaved. A version that was replaced quickly, or that ran with a high error rate, is not a candidate.
4. Choose the newest version that ran in production for at least 24 hours with normal error rate and latency.
5. Confirm that its configuration does not conflict with changes that have since been made to shared settings.

6. Write the chosen version and your reasoning in the incident channel before you run the command.

Observation	Meaning for selection
Status failed	Never a target
Status completed, short life, degraded metrics	Not a target
Status completed, long life, healthy metrics	Candidate
Newest candidate	The target

The previous version recorded on a deployment is only a starting point. It might itself have been unhealthy; always verify.

6.7 Configuration rollback

A bad setting can cause an incident even when the code is unchanged. Configuration is stored with the release, so a configuration problem is usually fixed by rolling back to the previous release, which restores the previous configuration with it. When only configuration has changed and the code is fine, apply the previous value through the configuration service, with the same approvals as a rollback.

Always compare the configuration diff of the suspect release against the dashboards. A setting that was lowered, such as a pool size or a timeout, often has an immediate and visible effect on saturation. After restoring a setting, confirm the effect on the relevant metric, not merely that the change applied.

6.8 Forward fix

A forward fix is a new release that corrects the defect while keeping the schema that the failed release introduced. It is used when rollback is unsafe, for example after a schema migration. A forward fix is a change like any other and goes through the canary, but the IC may approve an accelerated schedule. Because it is unproven, keep the incident open and the severity unchanged until the metrics have recovered.

6.9 Failed releases

When a release is marked failed, the deploy bot has already stopped promotion and traffic stays on the previous version. A failed release may still leave unhealthy instances behind. Confirm that all instances of the stable version are ready, remove leftover instances of the failed version, and check whether capacity is lower than normal. Record the cause on the ticket before any retry is planned.

7. Service-specific runbooks

7.1 payments-api

7.1.1 Purpose and dependencies

payments-api is owned by the payments team and depends on payments-db, ledger-service and fraud-scorer. It runs ten instances. Normal p95 latency is around 200 ms. The Payments on-call is the first contact.

7.1.2 Controlled setting: DB_POOL_MAX

DB_POOL_MAX is the maximum number of database connections each instance may hold. It is a controlled setting. In production DB_POOL_MAX must be 50, and the minimum allowed value is 30. Values below 30 are rejected by the pre-flight check. Any change requires load-test evidence attached to the release ticket. This rule is the action item that followed INC-0917, in which a pool reduction without load testing exhausted the pool.

7.1.3 Symptoms of pool starvation

When the pool is too small, requests wait for a connection. The service shows rising p95 latency, timeouts when acquiring a connection, rising orders-api retry queues and a growing number of pool waiters. Waiters above 20 for 5 minutes is SEV2.

Metric	Normal	Concerning
Pool in use	Moderate, well under the maximum	Pinned at the maximum
Pool waiters	0	Sustained above 20
p95 latency	Near 200 ms	Several times higher
Error rate	Under 0.2%	Rising

7.1.4 Diagnostic procedure

1. Open the payments-api dashboard and read pool in use and waiters.
2. Read the effective configuration of a running instance and confirm DB_POOL_MAX.
3. Compare with the configuration diff of the most recent deployment.
4. Check payments-db for saturation, so that you can tell a small pool from a slow database.
5. Check the logs for connection acquisition timeouts.

```
deployctl config payments-api --show DB_POOL_MAX
```

Expected output is a single line giving the effective value on the running version.

7.1.5 Remediation

If the pool setting is out of range and the release is within the rollback window, follow chapter 6. If a setting alone is wrong, restore the previous value through the configuration service. Do not raise the pool above the controlled value to compensate for a slow database; that moves the problem to payments-db, which has its own connection limits.

7.1.6 Fallback behaviour

If fraud-scorer is unavailable, payments-api falls back to the documented low-value acceptance rule. If ledger-service is unavailable, ledger-dependent calls fail and payments cannot be recorded. In either case, notify the owning team and the Incident Commander.

7.2 auth-service

7.2.1 Purpose and dependencies

auth-service is owned by the identity team. It depends on session-cache and identity-db. It runs six instances. Normal p95 latency is around 100 ms. The Identity on-call is the first contact.

7.2.2 Controlled setting: SESSION_TTL_SECONDS

SESSION_TTL_SECONDS controls how long a session lives. It must be between 900 and 7200, and the production value is 3600. The service validates the range at startup, following INC-0801, in which the value was entered in the wrong unit and shoppers were signed out within seconds. A value outside the range must be treated as a defect even if the service starts.

7.2.3 Symptoms of TTL misconfiguration

A too-short TTL expires sessions mid-flow, so shoppers are signed out while paying. The session cache sees keys expiring far faster than normal, the hit ratio falls, and every miss falls through to identity-db. Cache hit below 80% suggests TTL misconfiguration.

Metric	Normal	Concerning
Cache hit ratio	Mid-nineties percent	Falling below the line in 7.2.3
401 session expired errors	Rare	Frequent, mid-flow
identity-db connections	Steady	Surge of re-authentication
Key expiry rate in session-cache	Baseline	A large multiple of baseline

7.2.4 Diagnostic procedure

1. Read the cache hit ratio and the error rate on the auth-service dashboard.
2. Read the effective SESSION_TTL_SECONDS on a running instance.
3. Check the session-cache expiry rate and memory.
4. Check identity-db connection counts for a surge.
5. Compare with the latest deployment's configuration diff.

```
deployctl config auth-service --show SESSION_TTL_SECONDS
```

Expected output is one line with the effective value in seconds.

7.2.5 Remediation

If the setting is wrong and the release is recent, roll back as described in chapter 6, then confirm the hit ratio recovers. Expect recovery to take several minutes as sessions are rebuilt; a re-authentication surge can load identity-db temporarily, so watch its connections and warn the DBA lead if they approach the limit.

7.3 checkout-web

7.3.1 Purpose and dependencies

checkout-web is owned by the storefront team. It depends on payments-api, orders-api and auth-service. It runs eight instances and holds no database connections of its own.

7.3.2 Canary procedure

The canary for checkout-web uses the steps 5%, 25%, 50% and 100% of traffic. At each step the deploy bot waits for the observation period and evaluates readiness, error rate and latency. Three consecutive readiness failures abort the canary. When a canary aborts, traffic stays on the previous version and the failed instances are removed.

7.3.3 DNS pre-flight for new upstream hosts

A release that introduces a new upstream host needs a DNS pre-flight check. The check resolves the host from the target network and fails the release if resolution fails. This follows INC-0755, in which a canary stalled on a host that was not provisioned. If your release adds a host, ask the platform team to confirm the record before the pipeline starts, rather than waiting for the check to fail.

```
dig +short <upstream-host>
```

Expected output is one or more addresses. Empty output means the record does not exist and the release must not proceed.

7.3.4 Handling an aborted canary

1. Confirm that the stable version is serving traffic.
2. Count ready instances against the normal count for the service.
3. Remove or restart instances of the aborted version that are in a crash loop.
4. Record the failing readiness message and its cause on the ticket.
5. Do not retry the release until the cause has been fixed and, where relevant, the pre-flight check passes.

Capacity matters: with fewer ready instances than normal, checkout-web has less headroom for peaks. Tell the platform team and avoid other changes until capacity is restored.

7.4 orders-api

7.4.1 Purpose and dependencies

orders-api is owned by the orders team. It is a tier 2 service with six instances and a single database dependency, payments-db. The orders on-call is the first contact for service questions and the DBA lead for database questions.

7.4.2 Retry queue behaviour

When payment status lookups are slow, orders-api places the lookups in a retry queue. A short queue is normal. A queue that grows without bound means a dependency is slow or unavailable and that orders are being created without a known payment state.

Retry queue depth	Interpretation	Action
Low and stable	Normal	None

Retry queue depth	Interpretation	Action
Rising for several minutes	Dependency slow	Check payments-api and payments-db
Large and still rising	Dependency failing	Declare an incident, tell the Payments on-call

7.4.3 After a database event

orders-api holds database connections and therefore needs a restart after a database failover. Restart ordering is covered in chapter 8. After a restart, confirm that the retry queue drains and that no order remains stuck in an unknown payment state; reconciliation of any such order is performed by the orders team.

7.5 session-cache

7.5.1 Purpose and dependencies

session-cache is a three-node Redis deployment owned by the platform team. It has no dependencies. It holds session data for auth-service.

7.5.2 Health checks

Check	Healthy	Investigate when
Nodes up	Three of three	Any node down
Memory used	Comfortably below the limit	Approaching the limit
Evictions	Zero	Any sustained evictions
Key expiry rate	Near baseline	Far above baseline
Latency	A few milliseconds	Tens of milliseconds

Check a node directly with the Redis client.

```
redis-cli -h <node> info memory
```

Expected output includes used_memory_human and maxmemory_human. Evictions mean the cache is too small for the session volume; expiry rates far above baseline usually point to a session TTL issue in auth-service, so look there first.

7.5.3 Node failure

If one node fails, the cluster continues with reduced capacity. Sessions on the failed node may be lost, and affected shoppers sign in again. Replace the node through the platform team's procedure, then watch identity-db for a short-lived surge of sign-ins.

7.5.4 Full cache loss

If the whole cache is lost, every session must be rebuilt from identity-db. Warn the DBA lead before restoring the cache, consider rate limiting sign-in at the edge gateway to spread the load, and restore the cache before lifting limits. This is a Tier 1 situation and the Identity on-call and the platform team work it together.

8. Post-failover recovery and service restart sequencing

8.1 Why order matters

After a failover or a long database outage, services that hold connections to payments-db carry stale or broken connection state and must be restarted. Restarting them all at once creates a surge on a database that has only just been promoted, and starts services that depend on each other in the wrong order. The sequence in this chapter avoids both problems.

8.2 Restart order

The order is fixed. Restart one stage at a time and move to the next only when the current stage is healthy.

Step	Component	Notes
1	pgbouncer-payments	Reconnect the pooler to the new leader first
2	ledger-service	Must be healthy before payments-api starts
3	payments-api	Refuses to start without the ledger
4	orders-api	Resumes status lookups and drains retries
5	notification-service	Last, to avoid duplicate customer messages

For each stage, wait for the component to pass health checks for 2 minutes before starting the next one. A single passing check is not enough; the component must stay healthy for the whole period.

The reason ledger-service comes before payments-api is that payments-api refuses to start without ledger. Starting payments-api first would only produce a crash loop and delay recovery.

8.3 Procedure

1. Confirm that the new database leader is running and that replication state is understood.
2. Restart the pooler and confirm that it reports the new leader.
3. Restart ledger-service. Wait for its health checks to pass for 2 minutes.
4. Restart payments-api in rolling batches, not all at once. Wait for 2 healthy minutes.
5. Restart orders-api. Wait for 2 healthy minutes and watch the retry queue.
6. Restart notification-service last. Confirm that queued notifications are sent once and only once.
7. Announce in the incident channel that the sequence is complete.

Use the standard restart command for each component.

```
deployctl restart <service> --rolling
```

Expected output lists instances being restarted in batches and finishes with all instances ready.

8.4 Verification after each stage

Stage	Verify
Pooler	Reports the new leader; connections established
ledger-service	Health endpoint passing; no connection errors in logs
payments-api	Error rate and latency back in band; pool waiters near zero
orders-api	Retry queue draining
notification-service	Sending resumes without duplicates

8.5 If a stage will not become healthy

Do not start the next stage. Look at the logs of the failing component for connection errors, check that the pooler points at the right leader, and confirm that the database accepts connections for the service account. If the failure is not understood within 10 minutes, tell the Incident Commander and involve the owning team. Do not restart a component repeatedly in the hope that it will recover; each restart disturbs a database that is already under strain.

8.6 Services that are not restarted

Services that were not affected by the database event are left running, and restarting them adds risk without benefit. Chapter 5 explains which services are unaffected by a payments-db failover. Even for those, check their dashboards after the sequence ends, in case a shared dependency caused a secondary effect.

8.7 Closing out recovery

When all stages are healthy, confirm customer-facing behaviour with a test payment in the supported test mode, confirm that the standby is replicating again, and check that no queue is still holding backlog. Hand any reconciliation work to the owning teams and record it in the incident ticket.

9. Customer communications and SLA credits

9.1 Principles

Customers should hear from us before they have to ask. Updates are honest, specific and free of speculation. We say what is affected, what we are doing and when we will update next. We do not assign blame in public, we do not promise a fix time that we cannot support, and we do not share internal details such as host names or credentials.

9.2 Who communicates

The communications lead for the incident writes status page updates. Customer success owns direct contact with named customers, and support owns the ticket queue. All three use the same facts from the incident channel. Engineers do not reply to customers directly during an incident; they pass information to the communications lead.

9.3 Response times for customers

The first customer reply for a SEV1 must be sent within 1 hour of the customer's first contact or of the incident being declared, whichever comes first. The first reply acknowledges the problem, states the current scope and gives the time of the next update. Later replies follow the status cadence set in chapter 3.

9.4 Message structure

Part	Content
What happened	Plain description of the customer-visible effect
Who is affected	Scope in terms the customer understands
What we are doing	Current action, without internal detail
Next update	A specific time
What the customer can do	A workaround, if there is one

Avoid abbreviations, internal service names and technical jargon in customer messages unless the customer already uses them.

9.5 Handling escalated customers

An angry customer is often a customer with a real loss. Acknowledge the impact first, then give facts. Do not argue about the number of outages the customer remembers. Offer a written explanation after resolution and say when it will arrive. If the customer threatens legal action or contacts executives, tell the Incident Commander and customer success lead immediately so that the response is coordinated.

Customer messages may include personal data such as names, contact details and account references. Use them only to reply to that customer and do not paste them into shared engineering channels.

9.6 SLA tiers and credits

Service level commitments depend on the customer's contract tier. The Enterprise tier commits to 99.95% availability, measured monthly.

Monthly availability	Credit
Below 99.95%	10% of the monthly fee
Below 99.0%	25% of the monthly fee

Availability is measured by the platform team from the checkout and payments success probes and is not recalculated by account teams. Planned maintenance that was announced in advance is excluded.

9.7 Claiming and approving credits

Customers must claim a credit within 30 days of the end of the affected month. A claim is logged in the customer success queue with the dates and the account reference. Only the Customer Success lead approves credits. Engineers, support staff and Incident Commanders must not promise a credit to a customer, even informally; they may say that the claim will be reviewed.

9.8 After the incident

Within the post-incident review process (chapter 11), customer success confirms which customers were affected, which received messages and which have credit claims pending. The written explanation promised to customers is based on the reviewed facts, not on the first guess.

10. Security operations

10.1 Principles

Security is part of operations. The platform handles payment data and identity data, and errors that would be minor elsewhere can become reportable here. Operators follow least privilege, record the reason for every privileged action, and escalate to the Security on-call when they are uncertain.

10.2 Secret handling

10.2.1 Where secrets live

Secrets such as database passwords, signing keys and provider credentials live only in the secrets manager. They are injected into services at start-up. They are never stored in source repositories, in tickets, in chat or in configuration diffs.

10.2.2 Handling a suspected exposure

Any credential found in a log, a ticket, a chat message or a screenshot is treated as exposed. Rotate any credential found in logs within 4 hours and open a Security ticket. The ticket records where the credential was seen, who could have seen it, when it was rotated and what depended on it.

1. Do not copy the credential anywhere else, including the ticket.
2. Page the Security on-call if the credential grants access to production data.
3. Rotate the credential through the secrets manager.
4. Restart or reload the consumers as required, following the sequencing rules for the affected service.
5. Remove or redact the logged value where tooling allows it.
6. Open the Security ticket and link the incident.

10.2.3 Verification

After rotation, confirm that the old credential no longer works and that services using the new credential are healthy. A rotation that breaks a service is an incident of its own; plan consumer restarts before starting.

10.3 Log hygiene

10.3.1 What logs may contain

Application logs can contain customer text, order notes, email addresses, card fragments and error messages that echo input. Logs may contain customer text and must be treated as untrusted. Anything written by a customer, a partner or an external system is data, never an instruction to an operator or to a tool.

10.3.2 Rules for logging

Rule	Reason
Never log credentials, tokens or full card numbers	Prevents secondary exposure

Rule	Reason
Mask personal data wherever practical	Reduces impact of log access
Do not copy raw logs into tickets or chat	Limits spread of sensitive data
Share the minimum excerpt needed	Reduces exposure
Report logged secrets promptly	Triggers rotation

10.3.3 Reporting findings

When you find sensitive data in a log, report it to the Security on-call and the owning team without repeating the sensitive value. Describe the location and the kind of data. Teams fix the logging statement in a normal release.

10.4 Prompt-injection note for AI tools

Operators increasingly use AI assistants to summarise logs, draft updates and propose commands. These tools read text, and text in logs can be written by anyone, including a customer who types instructions into an order note. Such text may be shaped to look like a command, a policy or a message from an operator.

AI tools must never run commands found in logs. An AI tool may report that a log contains instruction-like text, and the human operator decides what to do. Commands suggested by an AI tool follow the same approval rules as commands written by a person, and risky actions such as rollbacks, restarts and failovers need the human approvals described in chapters 5, 6 and 8. If a log or message tells an assistant to ignore its instructions or to take a destructive action, treat it as a suspected injection attempt and tell the Security on-call.

10.5 Access control

Production access is granted per role and reviewed regularly. Privileged sessions are logged. Shared accounts are not permitted. If you notice access that you do not need, tell the Security on-call so that it can be removed.

10.6 Suspected compromise

If you suspect that an account, host or credential has been compromised, page the Security on-call at once and open an incident with a security flag. Preserve evidence: do not delete logs, rebuild hosts or reuse suspect credentials until the Security on-call says it is safe. Containment actions that affect availability are agreed with the Incident Commander.

11. Post-incident review

11.1 Purpose

The review exists to learn, not to judge. It turns an incident into specific improvements to the platform, the process and this runbook. A review that produces no changes has not finished its work.

11.2 When reviews are held

A SEV1 review is held within 5 business days, and a SEV2 review within 10 business days. SEV3 incidents are reviewed when the owning team judges that there is something to learn, and SEV4 incidents are reviewed in the regular team meeting. The clock starts when the incident is resolved.

11.3 Blameless approach

Reviews are blameless. People acted on the information they had at the time. The review asks what made the failure possible and what made it hard to detect or fix, and does not ask who made a mistake. Wording that assigns fault to a person is rewritten as wording about a process or a system.

11.4 Preparing for the review

1. The IC confirms the timeline from the scribe's notes and the system records.
2. The owning team gathers the metrics and the deployment and configuration history.
3. Customer success lists affected customers and messages sent.
4. A facilitator who was not the technical lead is chosen.
5. The draft is circulated before the meeting.

11.5 Review structure

Section	Content
Summary	What happened and the customer impact
Timeline	Key events in order, with times
Detection	How it was noticed, and how quickly
Response	What was tried and what helped
Root cause	The conditions that made it possible
What went well	Practices worth keeping
What went badly	Gaps in tooling, process or knowledge
Action items	Specific changes with owners and due dates

11.6 Action items

Action items need an owner and a due date. An item without both is not accepted into the review. Items are tracked in the owning team's backlog and reviewed at the platform team's weekly

meeting until closed. Open items from earlier incidents are examined at the start of each new review, because an unfinished item can turn into a repeat incident. An item that remained open after INC-0917 is a reminder of this.

11.7 Sharing the outcome

The final review is published internally. Customer success uses it to prepare the written explanation for affected customers. Changes to thresholds, procedures or contacts that arise from a review are made in this runbook through the usual pull request.

12. Glossary and reference tables

12.1 Glossary

Term	Meaning
Acknowledge	A named person accepts a page and says so in the incident channel
Canary	Gradual release to a small share of traffic before full rollout
Candidate	The standby node chosen to become the new leader
Controlled setting	A configuration value with a defined range and change rules
Forward fix	A new release that corrects a defect without reversing the old one
IC	Incident Commander
Leader	The Patroni role for the node accepting writes
Pool waiters	Requests waiting for a database connection
Replica	A node receiving replication from the leader
Replication slot	A marker that makes the database retain WAL for a consumer
RPO	Recovery point objective: acceptable amount of data loss
RTO	Recovery time objective: acceptable time to recover
Scribe	Role that records the incident timeline
SLA	Service level agreement with a customer
Switchover	Planned, controlled change of leader
Failover	Unplanned promotion of the standby
WAL	Write-ahead log

12.2 Service ownership

Service	Owning team	First contact
edge-gateway	platform-team	Platform on-call
checkout-web	storefront-team	Storefront on-call
auth-service	identity-team	Identity on-call
payments-api	payments-team	Payments on-call
orders-api	orders-team	Orders on-call
ledger-service	payments-team	Payments on-call
fraud-scorer	risk-team	Risk on-call
notification-service	platform-team	Platform on-call
session-cache	platform-team	Platform on-call
identity-db	dba-team	DBA lead

Service	Owning team	First contact
payments-db	dba-team	DBA lead

12.3 Historical incidents

These incidents shaped many of the rules in this runbook. They are cited so that readers understand why a rule exists.

Identifier	Date	Service	Title	Root cause	Outcome
INC-0917	2026-03-14	payments-api	Connection pool exhaustion after pool size change	Pool size reduced during a clean-up without a load test	Config rolled back; pool restored
INC-0862	2025-12-02	payments-db	Disk pressure from stale replication slot	Inactive analytics slot retained WAL	Slot dropped; checkpoint freed 160 GB
INC-0801	2025-09-21	auth-service	Login failures after TTL misconfiguration	Session TTL entered in the wrong unit	auth-service rolled back
INC-0755	2025-07-08	checkout-web	Canary stuck on missing DNS record	New upstream host not provisioned before release	Rolled back; DNS record created

12.4 Where each topic lives

Topic	Section
Severity and cadence	3.2 and 3.3
Contacts and escalation	4.2 and 4.3
Database health	5.1
Disk pressure	5.2
Replication slots	5.3
Switchover	5.4
Failover	5.5
Release approvals	6.2
Rollback	6.5
Service restart sequence	8.2
Customer credits	9.6 and 9.7
Secret exposure	10.2
Post-incident review	11.2

12.5 Quick command reference

Purpose	Command form
Cluster state	`patronictl list payments-db`
Planned role change	`patronictl switchover payments-db`
Release rollback	`deployctl rollback <service> --to <version>`
Rolling restart	`deployctl restart <service> --rolling`
Read a setting	`deployctl config <service> --show <KEY>`
Resolve a host	`dig +short <host>`
Cache memory	`redis-cli -h <node> info memory`

12.6 Closing note

This runbook is only useful if it is accurate. If a step fails, if an output differs from what is shown here, or if a contact has changed, correct it as soon as the incident is over. Every correction makes the next incident a little easier for the next person on call.